# CSE 554 Machine Learning Systems

## Lecture 2 – GPU Kernel Optimizations - 1

Baris Kasikci

BE BOUNDLESS

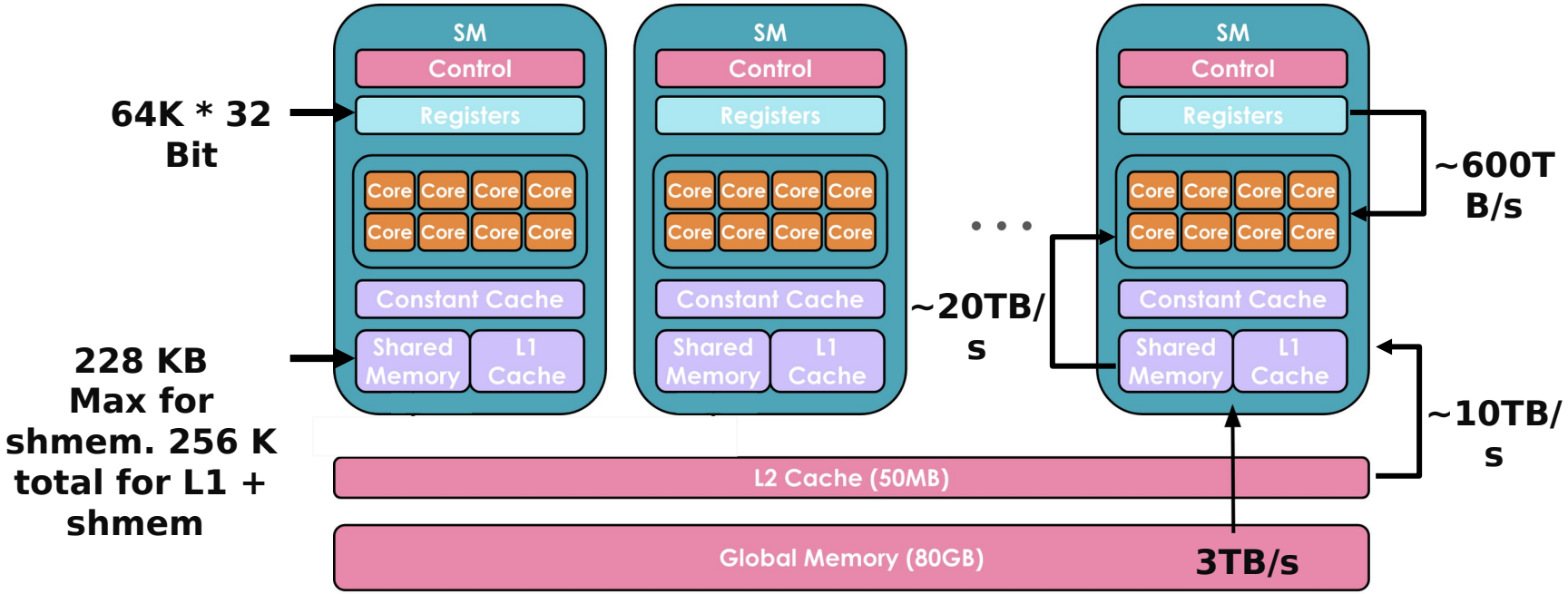
# Class Overview

1. GPU Architecture and Programming
2. GPU Kernel Optimizations
3. Transformers (Architecture and Implementation)
4. Performance Modeling
5. Quantization and Sparsity
6. Memory Management
7. Mixture of Experts (MoE)
8. Retrieval-Augmented Generation (RAG)
9. Batching
10. Speculative Decoding
11. Parallelism
12. End-to-end Serving
13. Agentic Systems
14. Large-scale Deployment
15. Agentic Systems
16. RL for LLMs

# Today

- Program the GPU using Triton and CUDA
- Learn the best practices of GPU programming
  - Coalesced loading
  - Using shared memory
  - Avoiding bank conflicts
  - Avoiding branch divergence
- Learn the state-of-the-art libraries
- Learn how to use Python bindings to expose kernels to Python

# Recap: GPU Architecture




https://nanotron-ultrascale-playbook.static.hf.space/assets/images/diving_primergpu2.svg

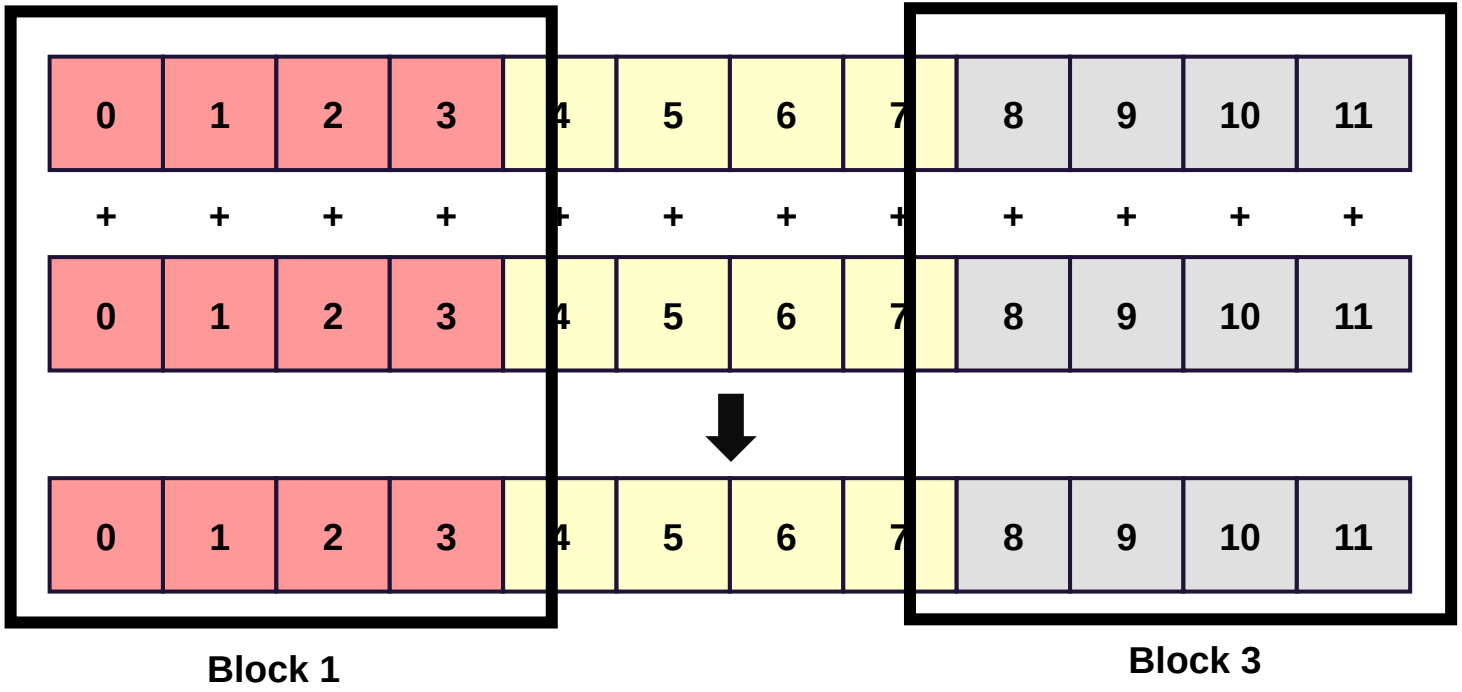
# Recap: GPU programming model

| Concept | Definition | Corresponding Architecture | Communication | Limits |
|---|---|---|---|---|
| **Thread** | Minimal units that execute instructions | Functional units | Local | Up to 255 registers |
| **Warp** | Group of Threads | SM tiles | Register File | 32 threads |
| **Thread Blocks** | Group of Warps | SM | Shared Memory | Up to 32 warps (1024 threads) |
| **Kernel** | Function on GPU | GPU | L2 / Global memory | Up to $(2^{32}-1)^3$ Blocks |

# What is Triton? Why Triton?

- Triton: A compiler framework from OpenAI for high-performance kernels with reduced human input
  - Python interface
  - Automated thread management
  - High performance
- Why Triton?
  - Write customized kernels easily
  - Higher performance than torch for complex kernels

# Partitioning the Addition Kernel



# Triton: Composing a Kernel at the Block Level

- Allocate tensors using torch
- Provides useful primitives
  - tl.load
  - tl.store
  - tl.min
  - …

```python
@triton.jit
def add_kernel(x_ptr,  # *Pointer* to first input vector.
               y_ptr,  # *Pointer* to second input vector.
               output_ptr,  # *Pointer* to output vector.
               n_elements,  # Size of the vector.
               BLOCK_SIZE: tl.constexpr,  # Number of elements each program should process.
               # NOTE: `constexpr` so it can be used as a shape value.
               ):
    # There are multiple 'programs' processing different data. We identify which program
    # we are here:
    pid = tl.program_id(axis=0)  # We use a 1D launch grid so axis is 0.
    # This program will process inputs that are offset from the initial data.
    # For instance, if you had a vector of length 256 and block_size of 64, the programs
    # would each access the elements [0:64, 64:128, 128:192, 192:256].
    # Note that offsets is a list of pointers:
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    # Create a mask to guard memory operations against out-of-bounds accesses.
    mask = offsets < n_elements
    # Load x and y from DRAM, masking out any extra elements in case the input is not a
    # multiple of the block size.
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    output = x + y
    # Write x + y back to DRAM.
    tl.store(output_ptr + offsets, output, mask=mask)
```

# What is CUDA

- Bare-bones
- One-to-one mapping to the hardware
- Potential for highest performance
- Heavy implementation burden
  - Need to think about offsets for each thread, etc.

# CUDA: Getting Full Control

- Memory allocation and deallocation
  - `cudaMalloc` -> device (GPU) memory allocation
  - `cudaMallocHost` -> pinned host (CPU) memory allocation
    - Will always be in memory and not swapped out. Gives you better performance when you move it to the GPU
    - Or you can use regular malloc
  - `cudaFree` -> free memory
- Memory movement and setting
  - `cudaMemcpy` -> synchronous copy
  - `cudaMemcpyAsync` -> asynchronous copy
  - `cudaMemset` -> synchronous set
    - E.g., similar to how you can fill a tensor with zeros in torch (torch.zeros_like())
  - `cudaMemsetAsync` -> asynchronous set

# CUDA kernels

- Declaring a kernel
  - `__global__ void kernel_name (args…)`
- Declaring a device “helper” function
  - `__device__ T helper_name (args …)`
- Args and return values are on the host
  - Pointers to device memory also reside in the host
- Inside a kernel, args (basic types, e.g. int) can be used and device pointers can be dereferenced
  - There are exceptions such as CUDA unified memory

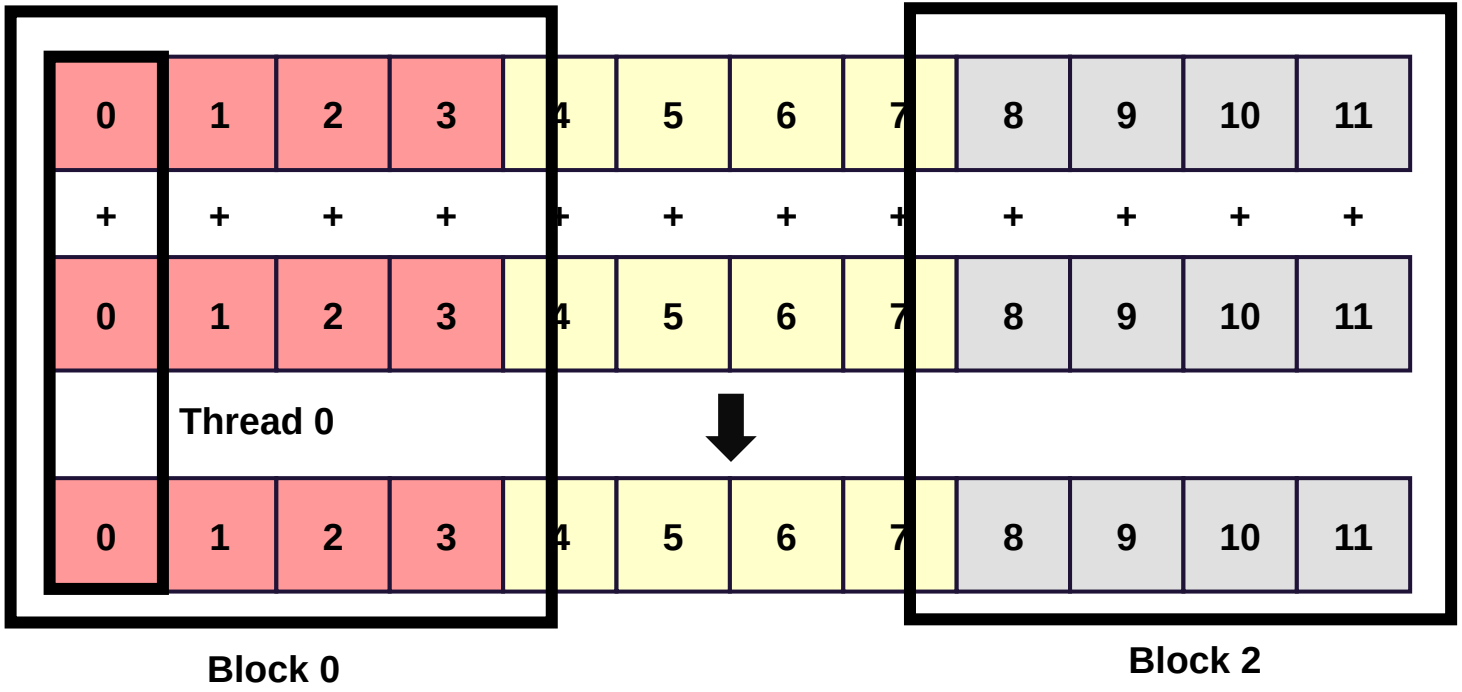
# Launching a Kernel

- Defining grid shapes (i.e., the block layout in a grid)
  - dim3 block(x,y,z)
- Defining thread shapes (i.e., thread layout per block)
  - dim3 thread(x,y,z)
- Launching kernels
  - kernel_name<<< block, thread >>> (args);

# Synchronization and Error checking

- Thread (Warp) synchronization
  - __syncthreads() -> device function
  - Useful when you use shared memory
  - Not needed within a warp (32 threads)
    - Recall that threads in warp execute in lockstep
- Block synchronization
  - Usually not feasible (you want blocks to operate independently), except for cooperative launch
- Device synchronization
  - cudaDeviceSynchronize() -> host function
    - CPU waits for the GPU
- Error checking (host function, error checking needs to be done explicitly)
  - cudaGetLastError()
  - cudaGetErrorString()

# Thread Assignment



# Writing a CUDA Addition Kernel

- Keep in mind: all blocks/threads execute the same code
- **Useful built-in variables**
  - `threadIdx`: index of the current thread within its block
  - `blockIdx`: index of the current block within the grid
  - `blockDim`: size of a block: number of threads in the block
  - `gridDim`: size of the grid: number of blocks in the grid

```
// Kernel function to add two vectors
__global__ void add(int *a, int *b, int *c, size_t num) {
    int block_start = blockIdx.x * blockDim.x;
    int thread_id = threadIdx.x;
    int index = block_start + thread_id;
    if (index < num) {
        c[index] = a[index] + b[index];
    }
}
```